GIT

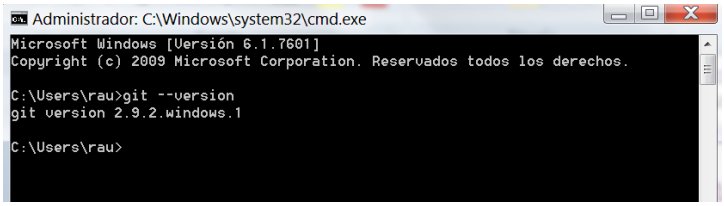
Git es un gestor de repositorios de versiones de Software. Antes de comenzar a explicar su funcionamiento aclarar que podemos encontrar:

- Documentación sobre Git
  - https://git-scm.com/book/es/v2
  - https://git-scm.com/documentation
  - https://www.atlassian.com/git/tutorials/setting-up-a-repository
- Información sobre la instalación de Git y el uso de GitHub
  - https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Instalaci%C3%B3n-de-Git
  - https://help.github.com/articles/set-up-git

Para saber si tenemos instalado Git iremos al terminal y escribiremos:

git --version

si Git está instalado obtendremos la versión del mismo:



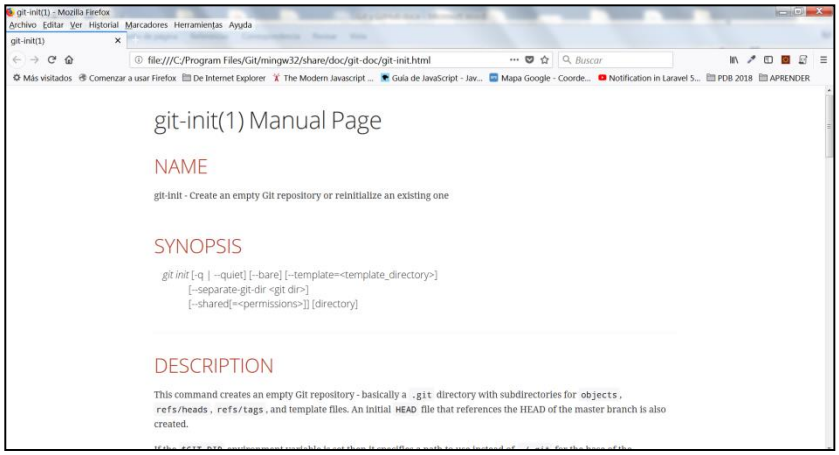
Realmente Git es un metacomando, el comando en sí es lo que escribimos justo después, p.e.:

git init : “init” es el comando u operación solicitada. En concreto “init” crea un repositorio de versiones para un proyecto dentro del directorio donde lo tecleemos.

Otros comandos disponibles para “git” que explicaremos más adelante son: “add”, “bisect”, “branch”, “checkout”, …

Si quisiéramos obtener una información resumida sobre algún comando usaremos la opción “--help". En Windows nos redireccionará a la documentación html que tenemos en nuestro equipo sobre dicho comando, p.e.:

git init --help



https://desarrolloweb.com/manuales/manual-de-git.html

Git fue creado para la colaboración de varias personas en un mismo proyecto, de ahí que cada acción que se realiza debe ir firmada por esa persona.

Para configurar las credenciales de cada usuario se usa el comando “config”:

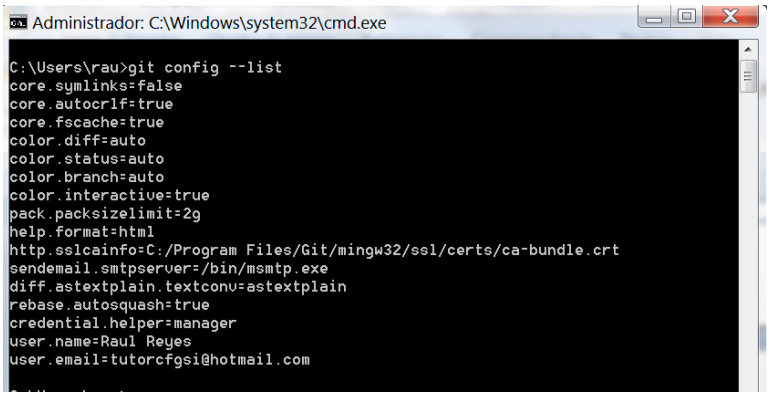
git config --global user.name “Raul Reyes”
git config --global user.email “tutorcfgsi@hotmail.com”

NOTA1: Esta información se puede redefinir para repositorios concretos, si se ejecuta el comando desde los mismos, pero sin la opción --global.

NOTA2: Esta información es independiente de las credenciales que se usan para conectarse al repositorio.

Para consultar las opciones configuradas usaremos:

git config --list



También podemos consultar una opción en concreto:

git config user.name

Git trabaja directamente con el directorio completo del Proyecto. Dentro de este directorio existen ficheros y subdirectorios que, por convención, tienen nombres ya preestableidos:

- README.md: Fichero de texto que contiene una breve descripción del Proyecto.
- LICENSE: Licencia de distribución del Proyecto.
- public: directorio para almacenar recursos web.
- bin: directorio para almacenar programas ejecutables.
- lib: directorio para almacenar las librerías usadas.
- test: directorio con las pruebas de funcionamiento correcto.

Las herramientas de gestión de proyectos también usan convención de nombres para ficheros y directorios:

- npm (Gestor de paquetes de Node.js)
    - package.json: información y dependencias
    - node_modules: dependencias instaladas
- Git (Gestor de versiones)
    - Directorio “.git”: grafo de cambios entre versiones
    - Fichero “.gitignore”: ficheros a ignorar

GITHUB

GitHub es un portal donde los programadores comparten repositorios con Proyectos Git. Permite el acceso tanto desde el navegador como a través de Git.

Para crear una cuenta iremos a la URL https://github.com

Cada una de las versiones que se crean del Software es denominada "**Commit**" o "Versión", que no son otra cosa que los ficheros de un Proyecto que son guardados en un repositorio y que pueden volver a restaurarse. Algunos Commits se etiquetan con tags especiales de versión, p.e. V1, V1.3, …

Una "**Rama**" es una secuencia de Commits ordenada por fechas donde cada nuevo Commit se añade al final. A la Rama principal se le llama "**Master**" y se crea automáticamente al crear el primer Commit.

Versión 1	Versión 2	Versión 3	Versión 4	Versión 5
A	A1	A1	A2	A2
B	B	B	B1	B2
C	C1	C2	C2	C3
↑	↑	↑	↑	↑
Commit 1 (Rama Master)	Commit 2	Commit 3	Commit 4	Commit 5

El "**Workspace**" o Directorio de Trabajo es el directorio local del S.O. donde se guardan todas las versiones del Proyecto.

El directorio de Commits "**.git**" se crea cuando ejecutamos "git init" y es un directorio oculto donde se añadirá el repositorio de Commits en una Base de Datos.

El "**Índice**" o Área de Cambios (STAGE) es el registro de cambios del Directorio de Trabajo que vamos a incluir en el próximo Commit, es decir, sólo irán en el siguiente Commit:
- los archivos que no se han modificado (por defecto) y
- aquellos cambios que registremos en el Índice.

1. *Para registrar los ficheros indicados ponemos:*

git add . → Registra todos los ficheros nuevos o modificados
git add LICENSE README.md → Sólo registra esos dos ficheros

Para extraer ficheros del índice escribimos:

git reset . → Extrae todos los ficheros del índice
git reset LICENSE → Sólo extrae ese fichero

2. *Para generar un nuevo Commit*

git commit --m "Mensaje" → Guarda el Commit con ese mensaje o titulo
git commit → Guarda el Commit y abre editor de texto para escribir mensaje
git commit --amend --m "Mensaje" → Modifica el último Commit con lo registrado en el Índice (cambia el "id" del Commit, con lo cual puede dar problemas si se ha distribuido el Commit)

https://desarrolloweb.com/manuales/manual-de-git.html

Cada Commit tiene un identificador que es único para cada uno que se crea y actúa como referencia única ya que no se repetirá en ningún repositorio.

El comando “git status” nos muestra el estado de los ficheros del directorio. Este puede ser:

- modified: Modificado respecto al Commit anterior
- untracked: No existente en el Commit anterior
- staged: Registrado para el próximo Commit

El comando “git diff” permite ver diferencias entre los ficheros con estado “modified”:

git diff: Muestra los cambios en todos los ficheros con estado “modified” del Directorio de Trabajo.
git diff README.md: Muestra los cambios de ese fichero sólo si su estado es “modified”.

Para los ficheros con estado “staged” usaremos:

git diff --cached: Muestra los cambios en todos los ficheros con estado “staged” del Directorio de Trabajo.
git diff --cached README.md: Muestra los cambios de ese fichero sólo si su estado es “staged”.

NOTA: “git diff” muestra las diferencias en las diferentes líneas de código indicando:

- Líneas añadidas → Color verde y comenzando por el signo “+”
- Líneas borradas → Color rojo y comenzando por el signo “-“
- El Contexto → Lo muestra en negro

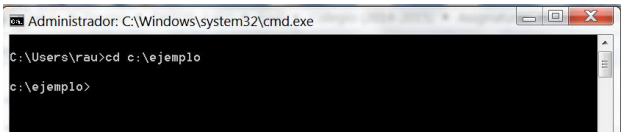
Otros comandos útiles son:

git mv file1.js file2.js: Cambia el nombre en el Directorio de Trabajo y en el Índice
git rm file1.js file2.js: Borra los dos archivos del Directorio de Trabajo y del Índice
git rm --cache <files>: Borra los ficheros del Índice cambiando su estado de “staged” a “untracked”
git checkout <files>: Borra los cambios de los ficheros pasando su estado a “unmodified”
git checkout . : Borra todos los cambios hechos en el Directorio de Trabajo

Vamos a realizar un ejemplo donde podremos ver, paso por paso, los resultados de los comandos vistos anteriormente:

1- Nos situamos en el directorio donde queremos crear el Directorio de Trabajo:

cd C:\ejemplo



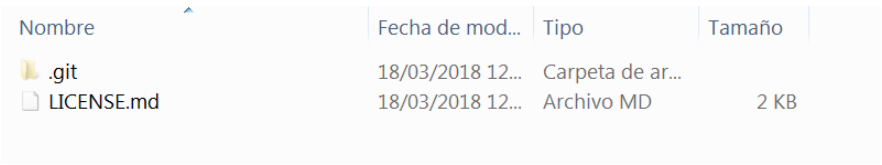
2- Creamos nuestro Repositorio de Versiones (observa que se crea el directorio ".git" con varios archivos y subcarpetas):

git init

3- Consultamos el estado de los archivos (observa que no hay resultado porque aún no tenemos archivos en nuestro repositorio):

git status -s

4- Vamos a crear y modificar el archivo "LICENSE.md"



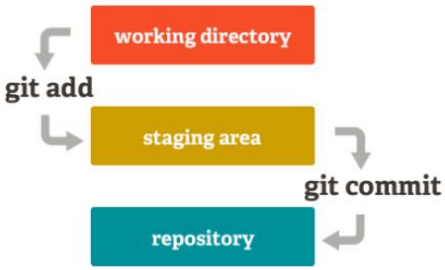
Nombre	Fecha de mod...	Tipo	Tamaño
.git	18/03/2018 12...	Carpeta de ar...	
LICENSE.md	18/03/2018 12...	Archivo MD	2 KB

5- Volvemos a consultar el estado de los archivos (observa que ahora aparece el archivo "LICENSE.md", con dos **??** de color rojo a su izquierda, porque ha sido modificado):

git status -s

```
Administrador: C:\Windows\system32\cmd.exe

C:\Users\rau>cd c:\ejemplo

c:\ejemplo>git init
Initialized empty Git repository in c:/ejemplo/.git/

c:\ejemplo>git status -s

c:\ejemplo>git status -s
?? LICENSE.md

c:\ejemplo>_
```

Disponemos de 3 espacios:



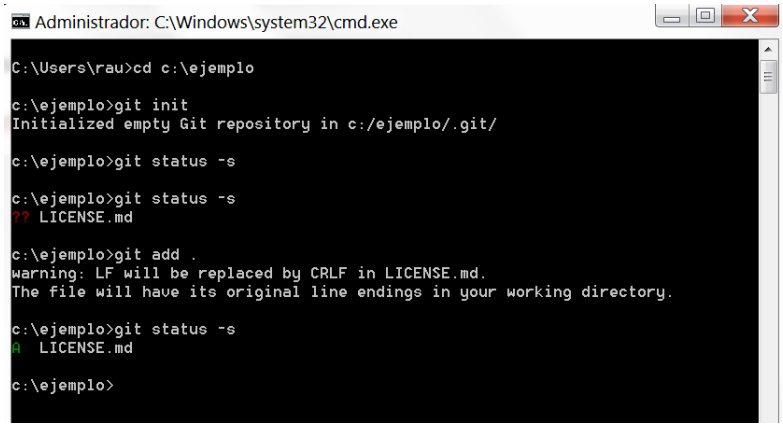
6- Registramos todos los ficheros nuevos o modificados al Índice (Stage):

git add .

```
Administrador: C:\Windows\system32\cmd.exe

C:\Users\rau>cd c:\ejemplo

c:\ejemplo>git init
Initialized empty Git repository in c:/ejemplo/.git/

c:\ejemplo>git status -s

c:\ejemplo>git status -s
?? LICENSE.md

c:\ejemplo>git add .
warning: LF will be replaced by CRLF in LICENSE.md.
The file will have its original line endings in your working directory.
```

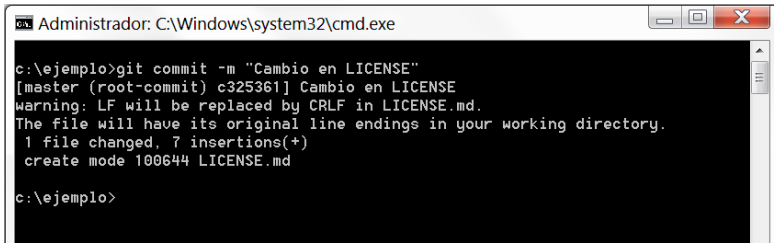
7- Volvemos a consultar el estado de los archivos (observa que ahora aparece el archivo "LICENSE.md", con una **A** de color verde a su izquierda, porque ha sido registrado):

git status -s



8- Realizamos un Commit con un Mensaje que incluirá, tantos los archivos no modificados (si los hubiera), como aquellos que han sido modificados, creados o eliminados que hayan sido registrados en el Índice. Ten en cuenta que al ser el primer Commit se creará la Rama Master:

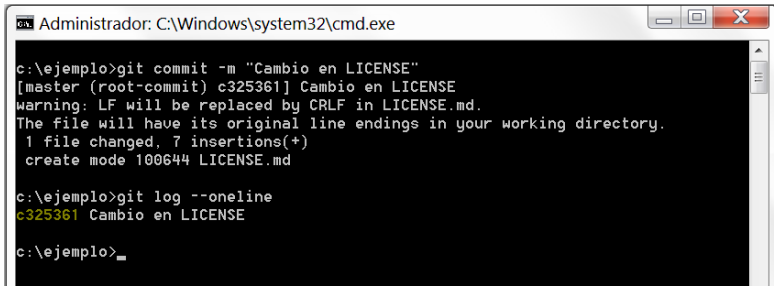
git commit --m "Cambio en LICENSE"



Si comprobásemos de nuevo el estado del repositorio (git status -s) podríamos verificar que no hay nada pendiente.

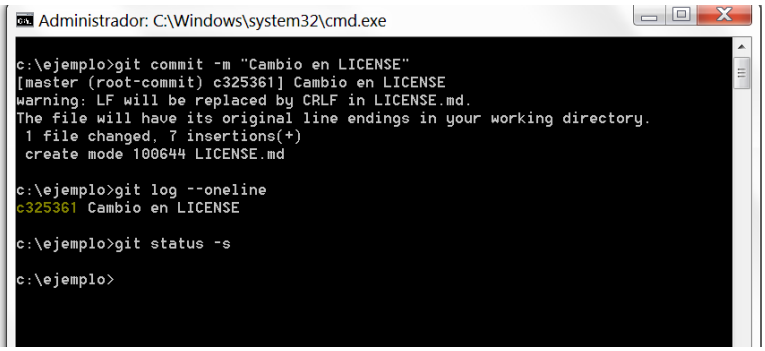
9- Ahora vamos a ver el Historial de Commits. En nuestro caso sólo aparecerá el Commit que acabamos de hacer:

git log --oneline



10- Si volvemos a consultar el estado de los archivos podemos ver que, de nuevo, no hay ningún archivo que se haya creado, eliminado o modificado desde el último Commit:

git status -s



11- Vamos crear en nuestro Directorio de Trabajo un nuevo archivo al que llamaremos "index.html" y que contendrá el siguiente código:

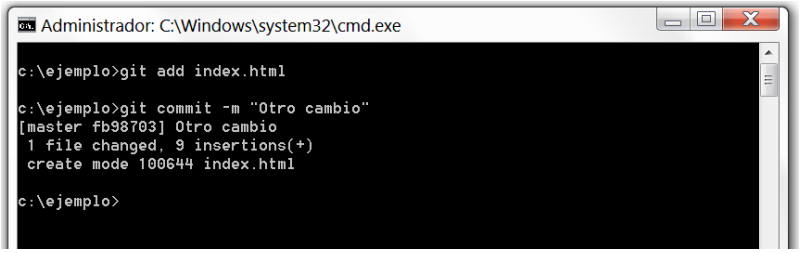
```
<!DOCTYPE html>
<html>
    <head>
        <title>Hola mundo</title>
    </head>
    <body>
        <h1>Nuevo fichero</h1>
    </body>
</html>
```

12- Vamos a registrar el nuevo fichero al Índice:

git add index.html

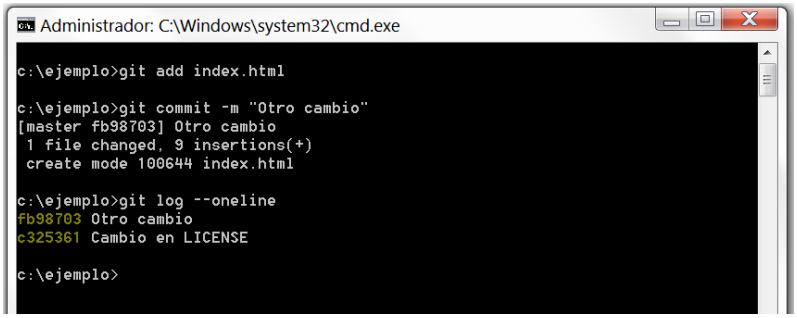
13- Realizamos un nuevo Commit con otro Mensaje:

git commit --m “Otro cambio”



14- Y volvemos a ver el Historial de Commits. Ahora aparecerán los dos Commit que hemos hecho:

git log --oneline



Para Git existen 2 tipos de repositorios:

1- Repositorios sin Directorio de Trabajo (Bare): sirven para compartir desarrollos con otros a través de internet. Suele estar alojado en un servidor de internet y se sincroniza con uno local mediante:
   a. **git push**: Sube ramas al repositorio "bare"
   b. **git clone**: Clona un repositorio en uno "local"
   c. **git fetch**: Trae ramas de un repositorio "bare" a uno "local"
   d. **git pull**: Integra una rama de otro repositorio con una rama del "local"

Este tipo de repositorios se crea mediante el comando:

git init --bare

Y crea un solo fichero con el repositorio de Commits y sin Directorio de Trabajo.

2- Repositorios con Directorio de Trabajo (Local o de Trabajo): se usa para desarrollar en el ordenador local. Git no permite hacer "push" en este tipo de repositorios.

Existen también portales web de repositorios como son GitHub o BitBucket que alojan repositorios compartidos y que permiten el acceso tanto desde la web como con los comandos Git.

Los repositorios en la nube se pueden identificar de varias maneras:

- https://github.com/tutorcfgsi/ejemplo
- https://github.com/tutorcfgsi/ejemplo.git
- git@github.com/tutorcfgsi/ejemplo.git

Desde GitHub vamos a poder:
- Crear un repositorio nuevo (New Repository)
- Copiar un repositorio de nuestra cuenta a otra cuenta (Fork)
- Importar un repositorio (Import Repository): realmente crea uno nuevo e importa
- Crear organizaciones para albergar varios proyectos relacionados
- Y las operaciones Git de sincronización de repositorios (push, clone, fetch, pull, …)

Para crear un repositorio en GitHub pulsaremos el botón '+' que aparece arriba a la derecha y seleccionamos "New Repository" teniendo que indicar un nombre para dicho repositorio.

Para subir un repositorio "local" al nuevo repositorio "bare" que hemos creado usaremos el comando "push" indicándole la rama donde queremos copiarlo:

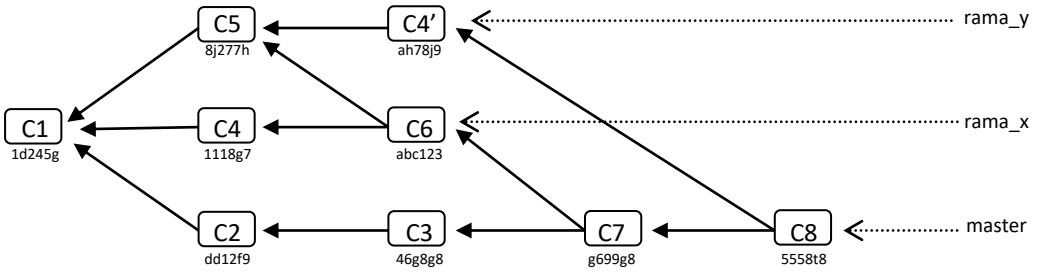
git push https://github.com/tutorcfgsi/ejemplo master

NOTA: Ten en cuenta que las ramas deben ser compatibles

Al hacer el "push" podemos observar que en GitHub aparecen el número de Commits que se han realizado.

RAMAS

La rama “master” es la rama principal de un repositorio y se crea al crear el primer Commit.



Observa que cada Commit está identificado por un valor (valor corto) que lo hace único y que el nombre de la Rama es un puntero al identificador de su último Commit:

- master → identifica al Commit **5558t8**
- rama_x → identifica al Commit **abc123**

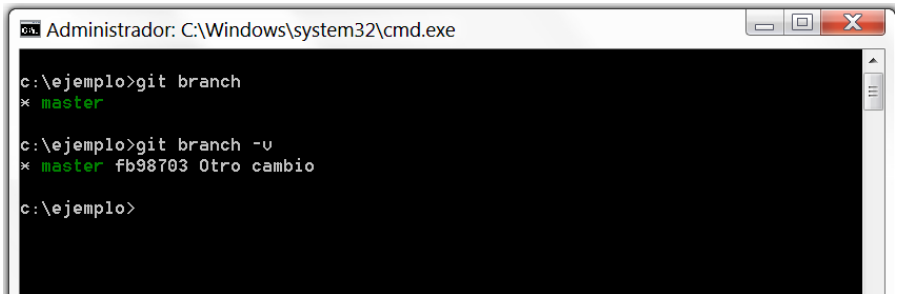
Para crear nuevas Ramas vamos a usar el comando “branch” indicándole el ID del Commit que queremos que sirva de base para la nueva Rama:

git branch rama_z 46g8g8

crea la Rama “rama_z” con base en C3 (46g8g8)

Para poder ver las ramas existentes usaremos también el comando “branch”. El argumento “-v” nos mostrará el comentario del último Commit realizado en dicha Rama:

git branch [-v]

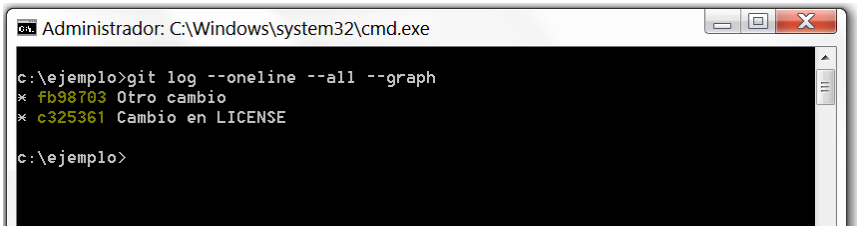


Si miramos el Grafo de Commits de más arriba, podemos ver que hay 2 tipos de Commits:

- **Con una sola flecha**: Indica que se han generado por modificación del Commit anterior (nuevos ficheros, ficheros eliminados o ficheros modificados).
- **Con dos flechas**: Indica que se han generado por integración de 2 Ramas.

Para poder ver el grafo de Commits completo usaremos:

git log --oneline --all --graph



Con Git podemos ver también las diferencias en los ficheros de 2 Commits. Para ello usaremos el comando "diff":

git diff rama_x master

Muestra las diferencias entre el último Commit de ambas Ramas.

git diff rama_x

Muestra las diferencias entre el último Commit de rama_x y el último Commit de la rama donde nos encontramos

git diff rama_x master LICENSE

Sólo muestra las diferencias en dicho archive entre ambas Ramas

Para poder restaurar un Commit en el Directorio de Trabajo usamos el comando "checkout":

git checkout rama_x

Este comando restaura la rama_x en el Directorio de Trabajo y actualiza el "HEAD", que es un puntero al Commit que se está restaurando en el Directorio de Trabajo.

También podemos restaurar una Rama a un número de Commits realizados en dicha Rama del servidor. Ten en cuenta que las diferencias se mantienen en el Directorio de Trabajo, con lo cual, si las añadimos al Índice y hacemos un Commit se añadirán todos en un solo Commit:

git reset master~2

El comando "reset hard" no guarda los cambios en el Directorio de Trabajo, con lo cual se perderían si no estuviesen guardados en otra Rama o en otro repositorio.